

PYTHON DAY 2

Variables, Data Types, Objects, Type Conversion, and User Input

Total time: 3 to 4 hours | **Level:** Beginner

Today's Goal

Yesterday you gave Python direct instructions. Today you learn how programs store and work with data. By the end of Day 2, you should be able to create variables, identify common data types, inspect values, convert between types, accept user input, and build an interactive command-line program.

1. Warm-up: Rebuild Yesterday From Memory - 15 minutes

Create warmup.py without opening yesterday's code. Print your name, current age, age after 5 years, and one arithmetic calculation.

```
print("Name: Preetham")
print("Age:", 28)
print("Age in 5 years:", 28 + 5)
```

2. Variables - 35 minutes

Create variables.py:

```
name = "Preetham"
age = 28
height = 5.6
is_learning_python = True

print(name)
print(age)
print(height)
print(is_learning_python)
```

Now change a value:

```
age = 28
print(age)
age = age + 1
print(age)
```

Research questions: What happens when `age = age + 1` executes? Is Python solving an algebra equation? What does the name `age` refer to before and after reassignment?

Important mental model: in Python, a variable name is better understood as a name bound to an object than as a physical box.

3. Data Types - 40 minutes

Learn these first: `int` for whole numbers, `float` for decimal numbers, `str` for text, `bool` for `True/False`, and `None` for the absence of a value.

```
name = "Preetham"
age = 28
weight = 92.5
is_engineer = True
future_value = None

print(type(name))
print(type(age))
print(type(weight))
print(type(is_engineer))
print(type(future_value))
```

Experiment with values that look similar but have different types:

```
a = 10
b = "10"

print(a + 5)
# Uncomment and inspect the error:
# print(b + 5)

print(type(a))
print(type(b))
```

4. Prediction Experiment - 20 minutes

Before running the code, write your prediction for every line in predictions.md. Then run it and compare your prediction with reality.

```
print("5" + "5")
print(5 + 5)
print("5" * 3)
print(5 * 3)
```

For every surprise, explain why it happened in your own words.

5. Type Conversion - 30 minutes

Programs often receive data in one type but need another. Test:

```
age_text = "28"
age_number = int(age_text)
print(age_number + 2)
print(type(age_number))

price = "99.5"
print(float(price))
print(str(100))
```

Deliberately create failures:

```
int("hello")
int("10.5")
```

Read the exception. Record what conversion Python attempted and why it failed.

Research: read the official documentation for int, float, str, and type. For each, write the input, output, and one example.

Official references: <https://docs.python.org/3/library/functions.html> and <https://docs.python.org/3/library/stdtypes.html>

6. User Input - 30 minutes

```
name = input("What is your name? ")
print("Hello", name)
```

Now inspect the type returned by input():

```
age = input("Enter your age: ")
print(type(age))
```

Then fix this program:

```
age = input("Enter your age: ")
print(age + 5)
```

Correct direction: convert the input to an integer before arithmetic.

```
age = int(input("Enter your age: "))
print(age + 5)
```

Research question: Why does input() return a string instead of automatically guessing the type? Write your own explanation.

7. Mini Project - Interactive Profile Calculator

Create profile_calculator.py. Ask the user for name, age, city, and a number representing years into the future. Print a clean summary including the user's future age.

Requirements: use input() at least four times, convert numeric input with int(), use descriptive variable names, and print a readable multi-line summary.

8. Research Experiment: Assignment and Object Identity - 25 minutes

Run this:

```
x = 10
y = x

print(x)
print(y)
print(x is y)
print(id(x))
print(id(y))
```

Then run:

```
x = 10
y = x
x = 20

print(x)
print(y)
```

Write a 5 to 10 line explanation answering: What changed? What did not change? What does assignment do? Do not worry about mastering every `id()` implementation detail today.

9. Debugging Lab - 20 minutes

Run each example separately and fix it:

```
name = "Python"
print(Name)
```

```
age = input("Age: ")
print(age + 10)
```

```
height = float("five point six")
```

For each error, record: 1) exception name, 2) line that caused it, 3) what Python expected, and 4) your fix.

10. Main Project - Personal Life Stats CLI

Build `life_stats.py`. Ask for name, age, weight, height, and goal weight. Calculate and display: current age, age after 1 year, age after 5 years, difference between current and goal weight, and the data types of at least three values using `type()`.

Bonus challenge: calculate BMI only after researching the formula yourself from a reliable source.

11. Knowledge Check

Q1. What is the difference between a variable name and the value it refers to?

Q2. Why does `input()` return a string?

Q3. What is the difference between `int` and `float`?

Q4. What is the result type of `10 / 2`?

Q5. Why does `'10' + '20'` differ from `10 + 20`?

Q6. What does `int('28')` do?

Q7. Why does `int('10.5')` fail?

Q8. What does `None` represent?

Q9. What does `type()` help you investigate?

Q10. What changed when `x = y` and then `x` was reassigned?

12. Day 2 Completion Checklist

- ☐ I can create variables with meaningful names.
- ☐ I understand basic assignment and reassignment.
- ☐ I can identify int, float, str, bool, and None.
- ☐ I can use type() to inspect values.
- ☐ I can predict simple number and string behavior.
- ☐ I can use int(), float(), and str().
- ☐ I understand that input() returns text.
- ☐ I can investigate TypeError and ValueError.
- ☐ I completed the Interactive Profile Calculator.
- ☐ I completed the Life Stats CLI project.
- ☐ I wrote research notes about assignment and object identity.

Suggested Study Schedule

15 min warm-up | 35 min variables | 40 min data types | 20 min prediction experiment | 30 min conversion | 30 min user input | 30 min mini project | 25 min research experiment | 20 min debugging | 45 min main project and review.

Official References

Python Tutorial: <https://docs.python.org/3/tutorial/>

Built-in Functions: <https://docs.python.org/3/library/functions.html>

Built-in Types: <https://docs.python.org/3/library/stdtypes.html>

Exceptions: <https://docs.python.org/3/library/exceptions.html>

Day 3 Preview

Next: operators, comparisons, boolean logic, and if / elif / else. Your programs will start making decisions instead of simply executing every line.